Non-Functional Requirements

how well the system must work, as numbers

Watch the 3-min video

In one line: non-functional requirements are the quality targets (latency, availability, consistency, durability, scale, cost); written as numbers, they decide the architecture.

1 Same features, different systems

Two systems can have identical features and still need completely different architectures. A personal blog and a bank both "store and show data"; what separates them is **how well** they must do it. Those are the non-functional requirements (NFRs), also called quality attributes:

Attribute	Question it answers	Example target
Scale	How much load?	10,000 RPS peak, 500 M users
Latency	How fast for one request?	p99 < 200 ms
Availability	How often up?	99.9% per month
Consistency	Must every read be the latest?	Strong for balances, eventual for likes
Durability	Can data ever be lost?	No acknowledged write lost
Security, cost, compliance	Who can access, at what price, under which rules?	Encrypted at rest, budget, GDPR

2 Turn every adjective into a number

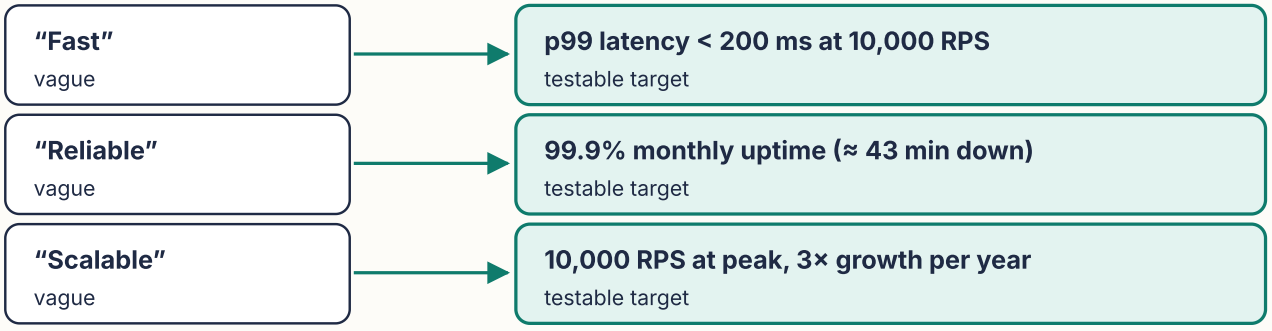
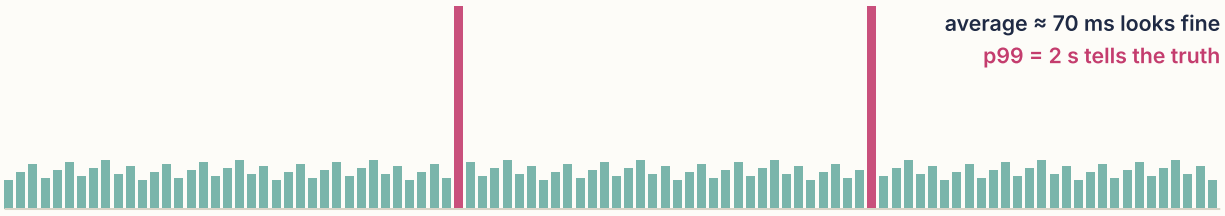


Figure 1. "Fast, reliable, scalable" mean nothing in a design review. A number can be tested and designed for.

3 Why p99, not the average



100 requests: most take ~50 ms (green), 1 in 100 takes 2 s (red)

Figure 2. The average hides the slow tail; p99 shows what 1 user in 100 waits.

A page often makes 20–50 backend calls, so a user hits the p99 latency of some call on almost every page load. That's why targets use percentiles (p50, p95, p99) measured at a stated load, not averages.

4 The nines, in downtime

Availability	Downtime per year	Per month	Typically needs
99%	~3.65 days	~7.3 h	One server, restarts by hand
99.9%	~8.8 h	~43 min	Redundant servers, health checks
99.99%	~53 min	~4.3 min	Multiple zones, automatic failover
99.999%	~5.3 min	~26 s	Multiple regions, heavy automation

Each extra nine costs much more to build and run, so ask what the business really needs.

5 Numbers change the architecture

- **High availability** → replicas across availability zones, automatic failover.
- **Low latency worldwide** → caches and a CDN near users, regional deployments.
- **Strong consistency** → a single source of truth or quorum writes; slower writes accepted.
- **High durability** → synchronous replication, backups, write-ahead logs.

WORKED EXAMPLE: A PAYMENT SYSTEM

The features are simple: charge a card, record the payment. The NFRs dominate: **strong consistency** (money is never double counted), **durability** (a confirmed payment is never lost), **idempotency** (a retried charge doesn't charge twice) and **auditability** (every change is traceable). Here correctness beats raw speed: 300 ms is fine, a lost payment is not.

COMMON MISTAKES

- Saying “fast and scalable” without numbers.
- Using averages for latency targets.
- Asking for five nines for an internal dashboard.
- Applying one consistency level to every feature.

6 Key terms

NFR / quality attribute: how well, not what

p99 latency: 99% of requests are faster

SLO: an internal target, e.g. 99.9%

SLA: a promise to customers, with penalties

Durability: acknowledged data is never lost

Idempotency: a retry has no extra effect

7 Interview questions

Q1. Which non-functional requirements change the architecture most? BASIC

Availability, latency, consistency, durability and scale, because each forces specific components: replicas and failover, caches and CDNs, quorums, synchronous replication, sharding.

Q2. How do you quantify “fast”? BASIC

As a percentile at a stated load: “p99 under 200 ms at 10,000 RPS”. Averages hide the slow tail that real users hit.

Q3. How much downtime does 99.9% allow? BASIC

About 43 minutes a month, or 8.8 hours a year. 99.99% allows about 4.3 minutes a month.

Q4. What NFRs would you set for a URL shortener's redirect path? SCENARIO

Very high availability (redirects are almost all traffic and links live in emails and printed media), low latency (p99 under ~100 ms), and high read scale; eventual consistency for a newly created link is usually fine.

Q5. Product says “it must never be down.” How do you respond? SCENARIO

Turn it into a number with a cost: 99.99% means multi-zone redundancy and automatic failover; 99.999% means multi-region and much more spend. Ask what an hour of downtime costs the business, then pick.

Q6. List the NFRs for a payment service. SCENARIO

Strong consistency, durability of confirmed payments, idempotent APIs, auditability, high availability for accepting payments, PCI-DSS compliance; moderate latency is acceptable.

Q7. Why can a service with p99 = 100 ms still feel slow? TRICKY

A page makes many calls, often in sequence. With 30 calls, the chance at least one hits its p99 is about 26%, and sequential calls add up. Use latency budgets per call and parallelise.

Q8. Can one system have different consistency targets? TRICKY

Yes, and it usually should. A shop needs exact stock and payments (strong) but can show reviews and recommendations a few seconds late (eventual).

Next: SD-A-005 · Trade-offs backendroadmap.com